

Reading Your Training Traces

A Fundamental Reference for the Filler-ASR SA Head

This document explains **every number** produced during training and validation of the filler-ASR self-attention head — from first principles ("what is a validation set?") to the statistical nuances of reading a metric as data. After reading it you should be able to open the training log or the wandb dashboard and **explain and act on every trace**: what it means, what it should and should not look like, and what it tells you about whether the model is learning, stuck, collapsing, over-fitting, or saturated.

It is written against the **actual `evaluate()` implementation** and uses the real numbers from the current run (`mask 20-30% + </s>x3 + fill_weight 0.1`, cold, 100 epochs).

FUNDAMENTAL REFERENCE · FILLER-ASR SA HEAD · TRAINING & VALIDATION TRACES

Part 0 — Fundamentals: training, validation, and what a "trace" is

What one training step does

Every optimizer step performs four operations:

1. **Forward** — audio is pushed through the model to produce per-frame logits `(B, T, 33)`.
2. **Loss** — a single number measuring how wrong those logits are versus the target.
3. **Backward** — the gradient: for every trainable weight, the direction that reduces the loss.
4. **Update** — AdamW nudges the ~157 M trainable parameters a small step (size set by the learning rate) in that direction.

This repeats ~109,800 times ($\approx 1,098$ steps/epoch $\times$ 100 epochs).

Train metric vs. eval (validation) metric — the core distinction

There are **two families** of numbers, and confusing them is the most common mistake.

	Train metric (<code>train/*</code>)	Eval / validation metric (<code>eval/*</code>)
Measured on	the current 32-clip batch being trained on	a fixed held-out set of 500 dev clips
Data status	seen, actively memorized	never trained on
Model mode	dropout ON , tap-masking ON (handicapped)	dropout OFF , masking OFF (<code>model.eval()</code>)
Sample	a different tiny batch every step (noisy)	the same 500 clips every time (stable)
Logged every	50 steps	2000 steps
Answers	"is optimization working?"	"is the model actually good / does it generalize?"

A model can score perfectly on data it memorized while being useless on new data. Only the **held-out** number tells you if it generalizes. Because eval runs with dropout and masking OFF, the model is evaluated at **full, deterministic capacity on clean input** — which is why the clean `eval` accuracy can be **higher** than the handicapped `train` accuracy.

What a "trace" is

A **trace** is one metric plotted over training steps — a **time series**. You never read a single value in isolation; you read its **trajectory** (falling? flat? spiking?) and its **relationship to the other traces**. A trace is data, and you analyze it like data: trend, noise, correlation, saturation.

Part 1 — What "eval" actually is (the implementation)

Every `eval/*` number comes from this one function (`train.py:evaluate`). It runs on **rank 0 only, over the full unsharded 500-clip dev set**.

```

@torch.no_grad()
def evaluate(model, loader, tok, fill_id, device, dump_k=10, max_sample_sec=10.0):
    was_training = model.training; model.eval()
    ...accumulators: w_err,w_tot,c_err,c_tot,s_err,s_tot, gc,gt,cc,ct,fc,ft,fp, ent_s,ent_n, predh, loss...
    for batch in loader:
        with autocast(bfloat16):
            lg = model(**inp).logits.float()
            m = min(lg.shape[1], labels.shape[1]); lg=lg[:, :m]; lb=labels[:, :m]
            loss_s += framewise_ce_loss(lg, lb, fill_id).item()
            pred = lg.argmax(-1); graded = lb!=-100; corr = (pred==lb)&graded
            gc += (corr).sum(); gt += graded.sum()
            isf = graded&(lb==fill_id); isc = graded&(lb!=fill_id)
            fc += (corr&isf).sum(); ft += isf.sum()
            cc += (corr&isc).sum(); ct += isc.sum()
            fp += ((pred==fill_id)&graded).sum()
            p = softmax(lg); ent = -(p*log p).sum(-1)
            ent_s += ent[graded].sum(); ent_n += graded.sum()
            predh += bincount(pred[graded])
            hyps = tok.batch_decode(pred, skip_special_tokens=True, group_tokens=False)
            for each utterance:
                ref = norm_ref(decode(labels[graded]))
                w_err += lev(ref.split(), hyp.split()); w_tot += len(ref.split())
                c_err += lev(ref, hyp); c_tot += len(ref); s_err += int(hyp!=ref); s_tot += 1
    return { "eval/loss": loss_s/loss_n, "eval/WER": w_err/w_tot, "eval/content_frame_acc": cc/ct, ... }

```

The mechanics that matter:

- `@torch.no_grad()` — no gradient graph (faster, less memory, cannot update weights).
- `model.eval()` — Dropout becomes identity **and** the train-only tap-masking is gated off. Eval is therefore **deterministic and unmasked**.
- **Accumulator pattern** — it sums counts over all 500 clips, then divides once at the end. So every `eval/*` is a **micro-average over the whole dev set** (long utterances contribute more frames/words), not an average of per-clip rates.
- **The eval loss is UNWEIGHTED** (no `fill_weight`) so the `eval/loss` curve stays comparable across weighted and unweighted runs.

Part 2 — Every trace, from basics to nuance

The train traces (every 50 steps, on the current batch)

Trace	What it is	Range / shape	Healthy	Red flag
<code>train/lr</code>	scheduler learning rate	triangle: 0→2e-4 over 5000 (warmup), then 2e-4→0	follows the schedule	(reference overlay, not a health metric)
<code>train/loss</code>	weighted CE on the batch	$\ln 33 = 3.5$ (random) → ~0.3	noisy but trending down	flat at 3.5, NaN/ exploding, wild oscillation
<code>train/graded_frame_acc</code>	batch frame accuracy (masked)	0.03 → high	rising	flat near 0.03
<code>train/grad_norm</code>	pre-clip global gradient L2 norm	your clip = 1.0	moderate, stable (yours ≈2-5)	spikes to 100s, →0, or NaN
<code>throughput/samples_per_s</code>	clips/s across 4 GPUs	~90 for you	steady	sudden drop = stall/ contention

Notes.

- `train/loss` is **weighted** (`fill_weight=0.1`) so its absolute scale is **not** comparable to `eval/loss` (unweighted) or to your old unweighted runs. Compare it only to itself over time. Jitter of ± 0.1 – 0.3 is normal (different tiny batch each step).
- `train/grad_norm` is your **optimization vital sign**. Occasional spikes caught by clipping are fine; persistent spikes mean instability (LR too high / outlier batches); a norm sitting **far above 1.0 for long** means most of the gradient is being clipped away each step (effectively a distorted, smaller step). $\rightarrow 0$ means dead.

The eval traces (every 2000 steps, clean held-out 500 clips)

Trace	Formula	Range	Random (eval@0)	Healthy target	What it tells you
<code>eval/loss</code>	mean unweighted CE	3.5 $\rightarrow$ ~ 0.3	3.42	falls then flattens	learning; rises = overfitting
<code>eval/graded_frame_acc</code>	gc/gt	0.03 $\rightarrow$ high	—	rising	inflated by easy <fill> — vanity metric
<code>eval/content_frame_acc</code>	cc/ct	0.03 $\rightarrow$ ~ 0.7 – 0.85	0.028	rising	the honest learning signal — are the characters right?
<code>eval/fill_frame_acc</code>	fc/ft	$\rightarrow \sim 1.0$	—	~ 1.0	recall of <fill>; low = boundary/leak problem
<code>eval/fill_pred_rate</code>	fp/gt	0 $\rightarrow$ 1	0.055	≈ 0.41 (true rate)	collapse detector — ~ 1.0 = predicts all fill
<code>eval/pred_entropy</code>	mean $-\sum p \cdot \log p$	$\ln 33 = 3.5 \rightarrow 0$	3.33	falls with accuracy	confidence / calibration
<code>eval/distinct_nonfill_tokens</code>	#classes predicted -1	0 $\rightarrow \sim 28$	23	climbs to ~ 26 – 28	vocabulary-collapse detector
<code>eval/WER</code>	<code>w_err/w_tot</code>	0 $\rightarrow$ >1	1.209	trending down	word error (uncapped, noisy — see caveat)
<code>eval/CER</code>	<code>c_err/c_tot</code>	0 $\rightarrow$ >1	8.53	down	char error (finer, less noisy)
<code>eval/SER</code>	<code>s_err/s_tot</code>	0 $\rightarrow$ 1	1.000	down	fraction of sentences not exactly right

The three metrics you must internalise:

- `content_frame_acc` answers "is it actually learning the transcript?" It starts at chance ($1/33 = 0.030$; your `eval@0 = 0.028` confirms "untrained") and rises toward the ceiling. **Watch this above all else.** If it climbs, the model is placing the right characters at the right frames. If it stays flat while loss falls, the model is only perfecting the trivial <fill> tail — the classic failure mode.
- `fill_pred_rate` should sit near the **true fill fraction ≈ 0.41** . The catastrophe is ~ 1.0 : the model predicts <fill> everywhere (minimises loss by saying "nothing") = **mode collapse**. If this climbs toward 1 while `content_frame_acc` craters, stop the run.
- `distinct_nonfill_tokens` should climb toward ~ 26 – 28 (the whole alphabet). Staying small (3–8) = **vocabulary collapse**: loss can look okay while the model cannot spell.

Two critical WER caveats:

1. **eval/WER is uncapped** — it decodes all frames, so any content leaking past `</s>` counts as insertions. That is why a random head scores $WER = 1.209 (>1)$. The **real** WER comes from `infer.py`, which caps decoding at the first `</s>`. Treat live `eval/WER` as a **pessimistic, noisy upper bound** that tightens as training improves; trust `content_frame_acc` (clean) and the eventual capped `infer.py` number.
2. **n = 500 → noisy**. Small wiggles between evals are sampling noise, not signal.

The samples table (qualitative). For the first 10 short dev clips it logs the raw per-frame prediction chain, the cleaned hypothesis, and the reference. Metrics are aggregates; the sample table shows you the **actual failure** — you can watch the left-packing form (`<fill> <fill>... → m i s t e r | ... → a crisp ... </s> </s> </s> <fill> ...` boundary) and diagnose why WER is high (smearing? repeats? wrong boundary? collapse?). **Read it every eval.**

Part 3 — Reading the traces together (the expert move)

No single metric is truth; you read the **joint pattern**. The healthy "it's learning" fingerprint is a **coordinated move**:

`eval/loss ↓ and content_frame_acc ↑ and pred_entropy ↓ and distinct_nonfill_tokens ↑ and fill_pred_rate ≈ 0.41 and WER ↓` — together.

The diagnostic power is in the **disagreements**:

Pattern	Diagnosis
loss ↓ but content_acc flat	learning the easy <code><fill></code> , not characters → target/architecture bottleneck
fill_pred_rate → 1 , content_acc ↓	mode collapse to fill — stop the run
content_acc ↑ but entropy → 0 too early	over-confident / mis-calibrated (confident wrong answers)
distinct small while loss ok	vocabulary collapse — cannot spell
train_loss ↓ but eval_loss ↑	over-fitting — generalisation ceiling passed
everything flat while lr still high	true model / architecture ceiling
everything flat only as lr → 0	the schedule ended it, not necessarily the ceiling
grad_norm spikes + loss jumps	instability — outlier batches / LR too high
samples show repeated chars / smeared boundary	the alignment-smearing failure (worst on long utts)

Part 4 — Traces as data: empirical & statistical analysis

Sampling noise (n = 500 dev is small)

Every eval metric is an **estimate** from 500 clips and carries error bars.

- **SER** is a proportion → binomial standard error $SE = \sqrt{p(1-p)/n}$. At $p=0.35$, $n=500$: $SE \approx 0.021 = 2.1\%$. A **±4% swing in SER between evals is noise**.
- **WER** is micro-averaged over words, but word errors are **correlated within an utterance**, so the effective sample size is closer to the **number of utterances (500)** than the number of words → roughly **±0.3-0.6% absolute** noise. Do not celebrate a 0.2% move between two evals.

- **The right tools:** put a CI on a WER by **bootstrapping over utterances** (resample the 500 clips with replacement $\sim 1000\times$, recompute WER, take 2.5/97.5 percentiles). To compare two runs/checkpoints, use a **paired sign test / McNemar** on per-utterance WER (exactly what the comparative-analysis report did). Comparing aggregate numbers without a paired test is how people fool themselves.

Signal vs. noise (time-series view)

- **Smooth before judging.** Use an EMA (wandb smoothing) or a 3-5-point moving average to see the trend through jitter. Read the smoothed line for direction, the raw line for stability.
- **Saturation = the per-eval delta $\rightarrow 0$.** Track $\Delta = \text{acc}(t) - \text{acc}(t-1)$. When Δ shrinks into the noise band **while lr is still meaningful ($> \sim 1e-4$)**, you have hit the ceiling. If Δ only shrinks as lr $\rightarrow 0$, the schedule is the limiter (a longer/cosine schedule might extract more).

Systematic bias (not just noise)

- `eval/WER` is **biased upward** (uncapped); the bias shrinks as fill prediction improves, so the curve falls faster than the true WER improves early, then converges to the capped number. Treat it as an upper bound that tightens.
- `eval/loss` (unweighted) vs `train/loss` (weighted) is a **definitional** offset — never compare their absolute values, only their trends.

Leading indicators

- `content_frame_acc` **leads** WER (frame accuracy improves before it shows up as fewer word errors) — use it as the early signal.
- `distinct_nonfill_tokens` and `pred_entropy` **lead collapse** — they move before WER blows up. Falling distinct / diving entropy while acc stalls = intervene early.

Part 5 — The ceiling: how good can this possibly get?

Three layers, hardest to softest:

1. **Architectural ceiling (hard).** The target pins char i to frame i — a **non-acoustic** placement. A frame-local classifier + 8 SA layers can only approximate it and **smears** on long utterances. This is why the mask baseline plateaus around **$\sim 10\%$ capped test-clean** and cannot reach the $\sim 2\%$ of full-rate acoustic SLAM-ASR. Only changing the target (CTC / CIF) breaks this.
2. **Empirical ceiling (your practical reference).** The **mask baseline's** final `content_frame_acc` / WER. **Overlay this run on that baseline in wandb.** "Am I learning enough?" literally means "is my curve at or above the baseline's?"
3. **Optimisation ceiling (soft).** Set by LR schedule, epochs, warmup. A plateau while lr is still high is a real limit; a plateau because lr decayed to 0 might yield to more epochs.

Meta-point: Stage-1 metrics are a **proxy**. The head feeds the LLM. The mask head scored $\sim 10\%$ standalone but drove **Stage-2 to 6.44%**. So the changes are ultimately judged by **Stage-2 decode WER**. Finishing move: pick the best checkpoint by **capped infer.py** (not the uncapped `best.pt` selector), then run it through Stage 2 and compare there.

Appendix A — The 30-second read of any eval line

Given `[eval @ N] WER=... content_acc=... fill_rate=... entropy=... distinct=...`:

1. **content_acc** vs last eval and vs baseline $\rightarrow$ is it learning / at the ceiling?

2. **fill_rate** near 0.41, not $\rightarrow 1 \rightarrow$ not collapsing?
3. **distinct** climbing toward $\sim 28 \rightarrow$ using the alphabet?
4. **entropy** falling in step with `content_acc` $\rightarrow$ confident for the right reasons?
5. **WER** trending down (remember: uncapped, noisy) $\rightarrow$ downstream proxy.
6. **the sample table** $\rightarrow$ what does the actual failure look like?
7. cross-check **lr** (plateau = model or schedule?) and **train vs eval loss** (over-fitting?).

Appendix B — Reference values (33-class vocab, $\sim 41\%$ fill)

Quantity	Random / floor	Perfect / ceiling
<code>content_frame_acc</code>	$1/33 = 0.030$	1.0 (architecture caps below this)
<code>pred_entropy</code>	$\ln 33 = 3.497$	0
<code>loss</code> (unweighted)	$\ln 33 = 3.497$	0
<code>fill_pred_rate</code>	~ 0.03 (uniform)	~ 0.41 (matches true rate)
<code>distinct_nonfill_tokens</code>	varies	~ 26 -28
WER / SER	$\geq 1 / 1.0$	0

Appendix C — This run's configuration

cold start (no warm-start) $\cdot$ 100 epochs $\cdot$ frozen HuBERT-xlarge tap $\cdot$ sinusoidal PE (scale 1.0) $\cdot$ 8
 post-norm SA layers $\cdot$ 25-frame duration budget $\cdot$ masking $U[0.20, 0.30] \times 10$ -frame spans $\cdot$ `</s>` $\times 3$ $\cdot$
`fill_weight` 0.1 $\cdot$ `lr` $2e-4$ $\cdot$ warmup 5000 $\cdot$ linear decay $\cdot$ clip 1.0 $\cdot$ effective batch 256 (32 $\times$ 2 $\times$ 4
 GPUs) $\cdot$ dev n=500 $\cdot$ eval/save every 2000 steps.